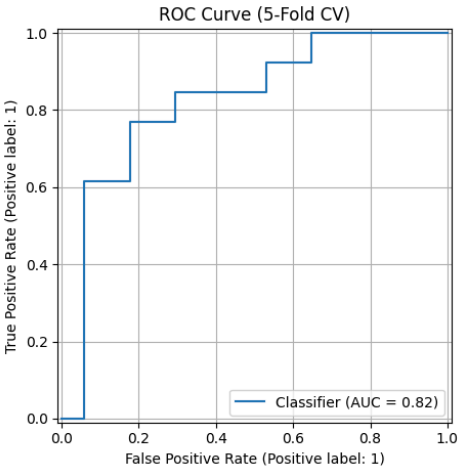
In [ ]:

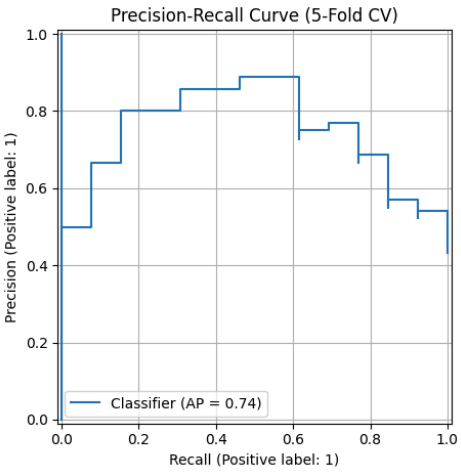
```
# 🎵 Maven Music Cancellation Predictor

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, cross_val_score, cross_val_predict
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score, RocCurveDisplay, PrecisionRecallDisplay
import joblib

# ----------------------
# 1. Load Data
# ----------------------
customers = pd.read_csv("maven_music_customers.csv")
xls = pd.ExcelFile("maven_music_listening_history.xlsx")
listening_history = xls.parse(0)
audio = xls.parse(1)
sessions = xls.parse(2)

# ----------------------
# 2. Clean Customers Table
# ----------------------
customers["Member Since"] = pd.to_datetime(customers["Member Since"])
customers["Cancellation Date"] = pd.to_datetime(customers["Cancellation Date"])

# Safely remove $ sign and convert to float
customers["Subscription Rate"] = (
    customers["Subscription Rate"]
    .str.replace(r"\$", "", regex=True)
    .astype(float)
)

# Fill missing plans with default
customers["Subscription Plan"] = customers["Subscription Plan"].fillna("Basic (Ads)")

# Fix outlier rate
customers.loc[customers["Subscription Rate"] > 20, "Subscription Rate"] = 9.99

# Discount to binary
customers["Discount?"] = np.where(customers["Discount?"] == "Yes", 1, 0)

# Clean email format
customers["Email"] = customers["Email"].str.replace("Email: ", "", regex=False)

# Add Cancelled column
customers["Cancelled"] = np.where(customers["Cancellation Date"].notna(), 1, 0)

# ----------------------
# 3. Clean & Merge Listening Data
# ----------------------
audio["Genre"] = audio["Genre"].replace("Pop Music", "Pop")

# Extract numeric Audio ID
audio["Audio ID"] = audio["ID"].str.extract(r"-(\d+)").astype(int)

merged = listening_history.merge(audio, on="Audio ID", how="left")
sessions_per_customer = merged.groupby("Customer ID")["Session ID"].nunique().rename("Number of Sessions")

# Genre dummy features
genres = pd.concat([merged["Customer ID"], pd.get_dummies(merged["Genre"])], axis=1).groupby("Customer ID").sum()
total_audio = merged.groupby("Customer ID")["Audio ID"].count().rename("Total Audio")

# Combine genre info
genre_df = genres.merge(total_audio, on="Customer ID")
genre_df["Percent Pop"] = genre_df.get("Pop", 0) / genre_df["Total Audio"] * 100
genre_df["Percent Podcasts"] = (
    (genre_df.get("Comedy", 0) + genre_df.get("True Crime", 0)) / genre_df["Total Audio"] * 100
)

# ----------------------
# 4. Create Model Dataset
# ----------------------
model_df = customers[["Customer ID", "Cancelled", "Discount?"]].copy()
model_df = model_df.merge(sessions_per_customer, on="Customer ID", how="left")
model_df = model_df.merge(genre_df[["Percent Pop", "Percent Podcasts"]], on="Customer ID", how="left")

# ----------------------
# 5. Train Model
# ----------------------
X = model_df.drop(columns=["Customer ID", "Cancelled"])
y = model_df["Cancelled"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

log_reg = LogisticRegression(max_iter=1000)
log_reg.fit(X_train, y_train)

y_pred = log_reg.predict(X_test)
y_proba = log_reg.predict_proba(X_test)[:, 1]

print("Classification Report:\n", classification_report(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("ROC AUC Score:", roc_auc_score(y_test, y_proba))

# ----------------------
# 6. Cross-Validation
# ----------------------
cv_auc = cross_val_score(log_reg, X, y, cv=5, scoring='roc_auc').mean()
cv_probs = cross_val_predict(log_reg, X, y, cv=5, method="predict_proba")[:, 1]

plt.figure(figsize=(10, 6))
RocCurveDisplay.from_predictions(y, cv_probs)
plt.title("ROC Curve (5-Fold CV)")
plt.grid(True)
plt.tight_layout()
plt.show()

plt.figure(figsize=(10, 6))
PrecisionRecallDisplay.from_predictions(y, cv_probs)
plt.title("Precision-Recall Curve (5-Fold CV)")
plt.grid(True)
plt.tight_layout()
plt.show()

# ----------------------
# 7. Key Insights
# ----------------------
fig, axs = plt.subplots(2, 2, figsize=(12, 10))

discount_df = model_df.groupby('Discount?')["Cancelled"].mean()
axs[0, 0].bar(['No Discount', 'Discount'], discount_df)
axs[0, 0].set_title("Cancellation Rate by Discount")

axs[0, 1].scatter(model_df["Number of Sessions"], model_df["Cancelled"])
axs[0, 1].set_title("Cancellations vs Listening Sessions")
```

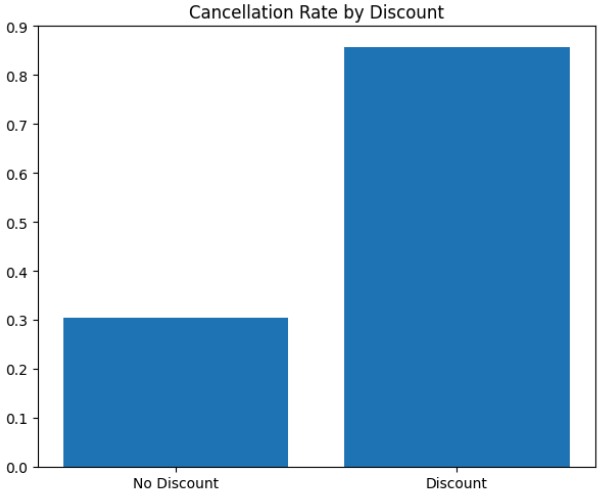
```python
axs[1, 0].scatter(model_df["Percent Pop"], model_df["Cancelled"])
axs[1, 0].set_title("Cancellations vs % Pop Music")

axs[1, 1].scatter(model_df["Percent Podcasts"], model_df["Cancelled"])
axs[1, 1].set_title("Cancellations vs % Podcasts")

plt.tight_layout()
plt.show()

# ----------------------
# 8. Export Model
# ----------------------
joblib.dump(log_reg, "cancellation_model.pkl")
```

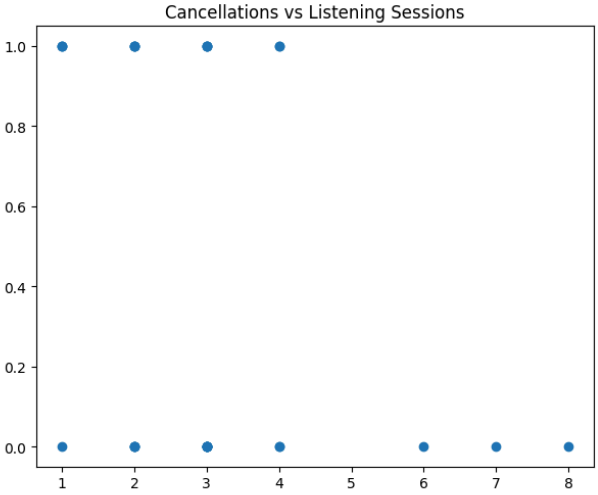
```
Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00         4
           1       1.00      1.00      1.00         2

    accuracy                           1.00         6
   macro avg       1.00      1.00      1.00         6
weighted avg       1.00      1.00      1.00         6

Confusion Matrix:
 [[4 0]
 [0 2]]
ROC AUC Score: 1.0
<Figure size 1000x600 with 0 Axes>
```

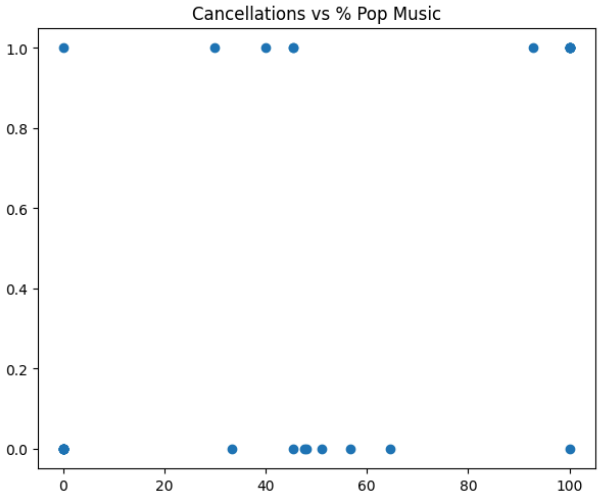


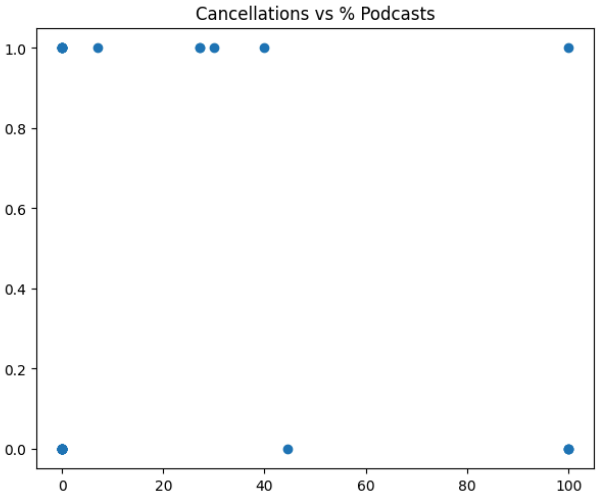

```
<Figure size 1000x600 with 0 Axes>
```











Out[ ]: ['cancellation_model.pkl']